Appunti del corso di
Basi di dati

UniVR - Dipartimento di Informatica

secondo semestre - A.A. 2025/26

Mattia Nicolis Matteo Drago

Indice

1 Tecnologie per le basi di dati **1**

1.1 Tecnologia dei DBMS 1

1.1.1 Transazione 2

1.1.2 Atomicità 3

1.1.2.1 Consistenza 3

1.1.2.2 Isolamento 4

1.1.2.3 Persistenza 4

1.2 Architettura di un DBMS 4

2 SQL **7**

2.1 Istruzione SELECT 8

1

Tecnologie per le basi di dati

Argomenti visti nel Modulo di Teoria:

- Progettazione concettuale
- Progettazione logica
- Linguaggi di interrogazione (algebra relazionale e calcolo relazionale)

1.1 Tecnologia dei DBMS

Un DBMS **(Database Management System)** o sistema di gestione di basi di dati è un sistema software in grado di gestire collezioni di dati che siano e **persistenti** assicurando la loro **affidabilità** e **privatezza**.

Un DBMS in quanto sistema software deve essere efficiente ed efficace e deve inoltre estendere le funzionalità del file system. Esempio: Gestione in memoria secondaria collezioni di dati.

Definizione: Basi di dati

Una base di dati è una collezione di dati gestita da un DBMS.

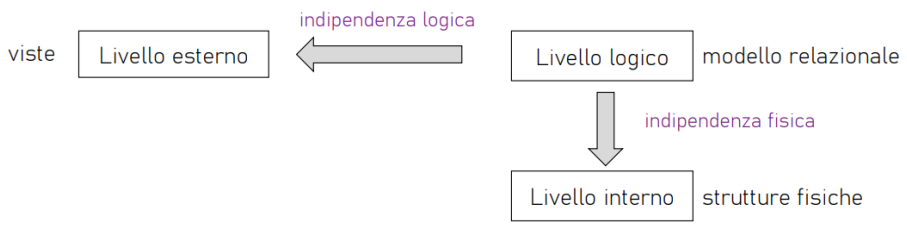


Figura 1.1: Architettura su 3 livelli del DBMS.

Un DBMS basato sul modello relazionale è nella maggior parte dei casi anche un sistema transazionale: fornisce un meccanismo per la definizione ed esecuzione di transazioni.

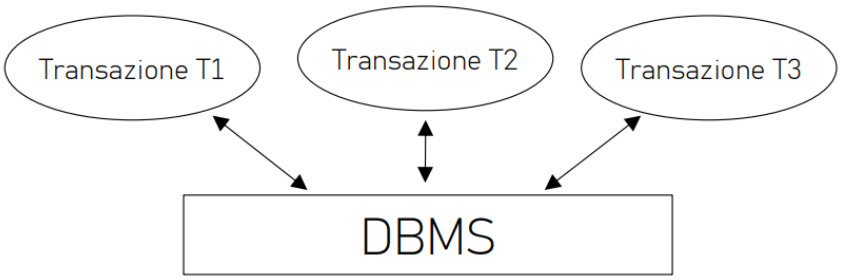


Figura 1.2: Ssistema transizionale

In sintesi, durante il corso si affronteranno i seguenti argomenti:

- Transazioni: definizione e proprietà
- Architettura di un DBMS:
  - Gestore delle interrogazioni: scrittura query in SQL
  - Gestore delle interrogazioni: ottimizzazione delle interrogazioni
  - Gestore dei metodi di accesso: indici primari e secondari, indici B+-tree e indici Hash
  - Gestore del buffer: interazione tra memoria secondaria e memoria centrale
  - Gestore della memoria secondaria: strutture fisiche per la memorizzazione dei dati in memoria secondari
  - Gestore della concorrenza: gestione dell'esecuzione concorrente (VSR, CSR, locking a due fasi)
  - Gestore dell'affidabilità: gestione dei guasti
- Architettura di un sistema document-based

1.1.1 Transazione

Definizione: Transazione

È un'unità di lavoro svolta da un programma applicativo (che interagisce con una base di dati) per la quale si vogliono garantire proprietà di correttezza, robustezza e isolamento.

Le caratteristiche principali sono:

- Una transazione o va a buon fine e ha effetto sulla base di dati o abortisce e non ha nessun effetto sulla base di dati.
- O tutto o niente! Non sono ammesse esecuzioni parziali

Sintassi per definire una transazione in SQL:

<transazione> → begin transaction

<programma>

end transaction

<programma> → ⟨ <istruzione> | commit work | rollback work ⟩
{ ⟨ <istruzione> | commit work | rollback work ⟩}

La transazione va a buon fine all'esecuzione di un commit work ma non ha alcun effetto se viene eseguito un rollback work.

Una transazione è ben formata se:

- Inizia con un begin transaction.
- Termina con un end transaction.
- La sua esecuzione comporta il raggiungimento di un commit o di un rollback work e dopo il commit/rollback non si eseguono altri accessi alla base di dati.

Esempio di transazione ben formata:

```
begin transaction
    update CONTO set saldo = saldo - 1200
        where filiale = '005' and numero = 15;
    update CONTO set saldo = saldo + 1200
        where filiale = '005' and numero = 205
    commit work;
end transaction
```

Proprietà delle transazioni

Una transazione ha quattro proprietà:
- Atomicità
- Consistenza
- Isolamento
- Persistenza

1.1.2 Atomicità

Una transazione è una unità di esecuzione **indivisibile**. O viene eseguita completamente o non viene eseguita affatto.

Implicazioni:

- Se una transazione viene **interrotta prima** del commit, il lavoro fin qui eseguito dalla transazione deve essere disfatto ripristinando la situazione in cui si trovava la base di dati prima dell'inizio della transazione.
- Se una transazione viene **interrotta dopo** l'esecuzione del commit (commit eseguito con successo), il sistema deve assicurare che la transazione abbia effetto sulla base di dati.

1.1.2.1 Consistenza

L'esecuzione di una transazione non deve violare i **vincoli di integrità**.

Implicazioni:

Verifica immediata :
- Fatta nel corso della transazione
- Viene abortita solo l'ultima operazione e il sistema restituisce all'applicazione una segnalazione d'errore
- L'applicazione può quindi reagire alla violazione.

Verifica differita :

- Al commit se un vincolo di integrità viene violato la transazione viene abortita senza possibilità da parte dell'applicazione di reagire alla violazione

1.1.2.2 Isolamento

L'esecuzione di una transazione deve essere **indipendente** dalla contemporanea esecuzione di altre transazioni.

Implicazioni:

- Il rollback di una transazione non deve creare rollback a catena di altre transazioni che si trovano in esecuzione contemporaneamente.
- Il sistema deve regolare l'esecuzione concorrente con meccanismi di controllo dell'accesso alle risorse

1.1.2.3 Persistenza

L'effetto di una transazione che ha eseguito il commit non deve andare perso. Implicazioni:

- Il sistema deve essere in grado, in caso di guasto, di garantire gli effetti delle transazioni che al momento del guasto avevano già eseguito un commit.

1.2 Architettura di un DBMS

L'architettura mostra i moduli principali che possiamo individuare nei DBMS attuali, considerando le diverse funzionalità che il DBMS svolge durante l'esecuzione delle transazioni. Per ogni modulo dell'architettura presentiamo le funzionalità che esso svolge e alcune delle tecniche che applicativo

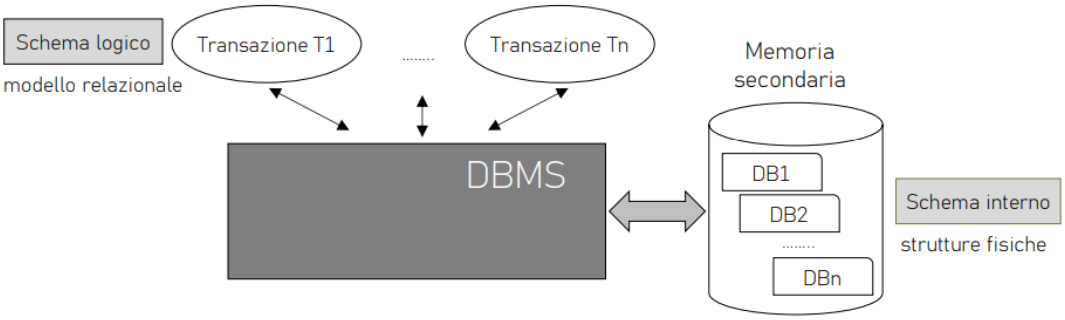


Figura 1.3: Architettura di riferimento di un DBMS.

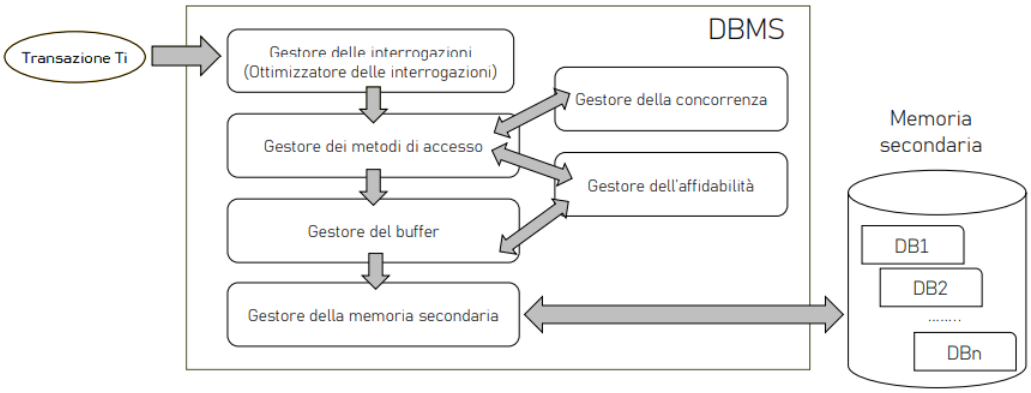
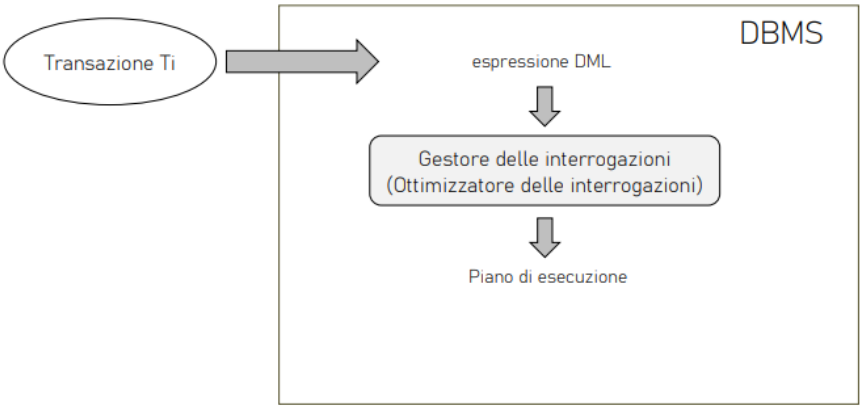
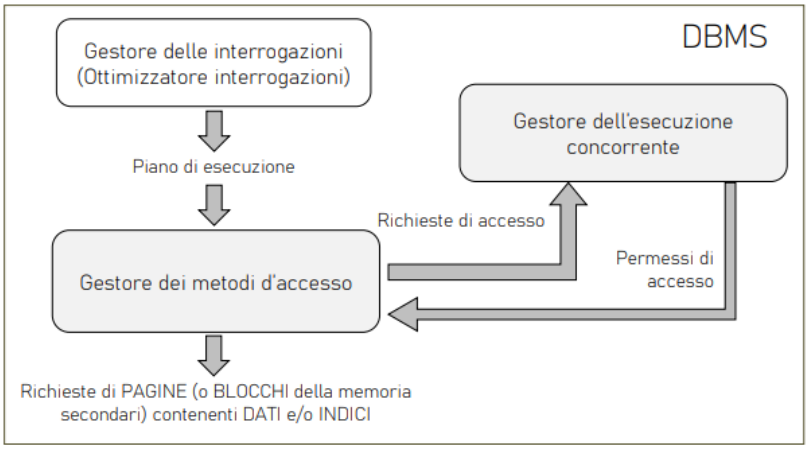


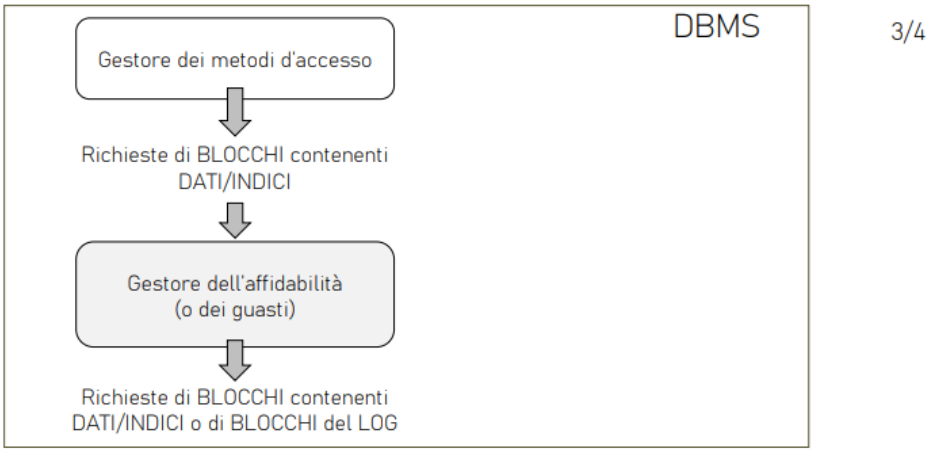
Figura 1.4: Architettura di riferimento di un DBMS.

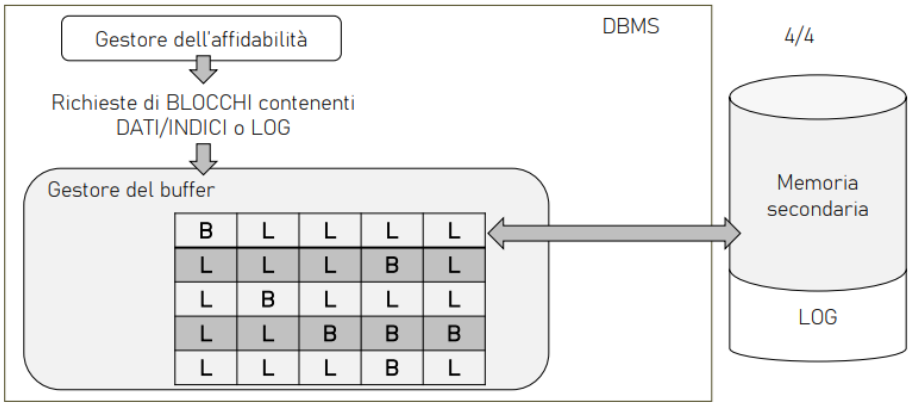


1/4



2/4





Quali modulo contribuiscono a garantire le proprietà delle transizioni?

Gestore dei metodi d'accesso: Consistenza
Gestore dell'esecuzione concorrente: Atomicità e Isolamento
Gestore dell'affidabilità: Atomicità e Persistenza

2

SQL

Il **termine SQL** in origine era l'acronimo di **S**tructured **Q**uery **L**enguagge, ma ad oggi è diventato un nome proprio.

SQL è un linguaggio che permette di:

- **definire delle strutture dati e i vincoli di integrità**: Data Definition Lenguagge
- **manipolare i dati** (inserimento, aggiornamento e cancellazione): Data Manipilation Lenguagge
- **interrogare**: Data Query Lenguagge

Esistono diverse versioni:

Nome informale	Nome ufficiale	Caratteristiche
SQL base	SQL-86	Costrutti base
	SQL-89	Integrità referenziale
SQL-2	SQL-92	Modello relazionale Vari costrutti nuovi 3 livelli: entry, intermediate, full
SQL-3	SQL:1999	Modello relazionale a oggetti Organizzato in diverse parti Trigger, funzioni esterne, ...
	SQL:2003	Estensioni del modello a oggetti Eliminazione di costrutti non usati Nuove parti: SQL/JRT, SQL/XML, ...
	SQL:2006	Estensione della parte XML
	SQL:2008	Lievi aggiunte (per esempio, `trigger instead of`)
	SQL:2011	Ricco supporto per dati temporali
	SQL:2016	Gestione del formato JSON

La parte di SQL dedicata alle dichiarazioni fa parte dei DML.
Le dichiarazioni vengono espresse in modo dichiarativo: specifica l'obiettivo e non la procedura per ottenerlo (algebra relazionale).

2.1 Istruzione SELECT

```
SELECT [ DISTINCT ]
[ * | expression [[ AS ] output_name ] [, ...] ]
[ FROM from_item [, ...] ]
[ WHERE condition ]
[ GROUP BY grouping_element [, ...] ]
[ HAVING condition [, ...] ]
[ { UNION | INTERSECT | EXCEPT } [ DISTINCT ] other_select ]
[ ORDER BY expression [ ASC | DESC | USING operator ]]
```